

Lab 1 instructions — simple edition (OPIM 5512)

Module 1 • keep this open during the lab. Every click you need tonight, in the order you need it. The data is already clean. Tonight is about the **workflow**: branch → commit → push → pull request → review → merge.

🚫 **No AI tonight.** Your only coding task is a three-line histogram. Type it. Module 3 is the AI unit.

Where every file goes

Grey = already in the repo when you click 'Use this template'. Purple = the only things YOU add tonight.

```
your-repo/
|-- README.md           data dictionary (given)
|-- REPORT.md           you fill the arrow lines at the end
|-- data/clean/
|   |-- weather_hourly_hartford.csv
|   |-- weather_hourly_stamford.csv
|   `-- demand_hourly.csv
|-- images/              <-- drag your PNGs here
|   |-- weather_line.png (A)
|   |-- weather_hist.png (A)
|   |-- demand_line.png  (B)
|   `-- demand_hist.png  (B)
`-- notebooks/           <-- Colab File > Save lands here
    |-- Lab1_A_Weather.ipynb (A)
    |-- Lab1_B_Demand.ipynb  (B)
    `-- Lab1_Joint_Optional.ipynb
```

How things get there

Notebook > notebooks/

In Colab: File > Save (plain Save; not Ctrl+S).
Repo = yours | branch = dev-weather / dev-demand
Keep the same path every save.

PNGs > images/

Colab downloads them to your Downloads folder.
GitHub Desktop > Repository > Show in Explorer
Drag into images/. If the name has (1) in it,
rename it back first - the report links to exact names.

Then, in GitHub Desktop

Top bar must say your dev- branch (not main).
Commit with a real message > Push >
github.com: Compare & pull request.

Before 5:30 — check three things

1. **GitHub Desktop opens and you're signed in.** File → Options → Accounts (Windows) / GitHub Desktop → Settings → Accounts (Mac). You should see your username.
2. **colab.research.google.com opens** and you're signed into a Google account.
3. **You know your campus word:** hartford or stamford.

Part 1 — Build the shared repo (Partner A drives, ~10 min)

Only **one** of you does this. Partner B watches — you'll need to know it too.

1.1 Make a new repo — from scratch

The point of tonight is your *first commit*, so you build the repo yourself. On **github.com**, click **+** (top right) → **New repository**. (Do NOT use a template — build it.)

Field	What to put
Owner	you (your personal account)
Repository name	opim5512-lab1-<netidA>-<netidB>
Visibility	Public (raw file links need it; there's nothing secret)
Initialize	✓ Add a README · ✓ Add .gitignore → Python

Click **Create repository**. You now have a clean repo with a README and a Python `.gitignore`. Everything else — the folders, the notebook, the plots — **you'll add tonight through the workflow**. That's the whole point.

1.2 Add your partner

Repo **Settings** → **Collaborators** → **Add people** → type Partner B's GitHub username → **Add**. **Partner B**: accept the invite — check your email, or the 🔔 bell on github.com. Until you accept, you can't push.

1.3 Protect `main`

Repo **Settings** → **Rules** → **Rulesets** → **New ruleset** → **New branch ruleset**: - Name: `protect main` · Enforcement: **Active** - Target branches → **Add target** → **Include default branch** - Rules: ✓ **Require a pull request before merging** → **Required approvals: 1** - Leave *Block force pushes* checked (it usually is) → **Create**

This is the two-person gate: from now on nothing lands on `main` without your partner's approval, and **you can't approve your own pull request**. (*Rehearsing solo with one account? Set Required approvals to 0.*)

1.4 Both partners: clone it — once

GitHub Desktop → **File** → **Clone repository** → **GitHub.com tab** → find the repo (click the 🔄 refresh icon if it's not listed yet) → note the **Local path** → **Clone**.

This is the *only* time you clone. From here on, GitHub Desktop **Pulls** new work down and **Pushes** yours up. The **Changes** panel shows *what changed*, not your files — to see files, use **Repository** → **Show in Explorer**.

1.5 Each partner: make your branch

Current branch → **New branch** → name it `dev-weather` (A) / `dev-demand` (B) → **Create branch** → **Publish branch** (the button top-right, or the big card in the middle).

● Read the top bar before every commit tonight. If it says `main`, stop and switch.

Part 2 — Plot & ship (each partner, on your own branch, ~30 min)

2.1 Open the starter notebook in Colab

The starter notebooks live in the **class repo**. In **colab.research.google.com** → **File** → **Open notebook** → **GitHub tab**, paste: `https://github.com/drdave-teaching/OPIM5512-labs` then open (under `Module1/Week1_TechStack/Lab1_FirstCommit/notebooks/`): - Partner A → `Lab1_A_Weather.ipynb` - Partner B → `Lab1_B_Demand.ipynb`

Open it in its **own tab** so these instructions stay open beside it.

2.2 Run it

- **Partner A:** set `CAMPUS = "hartford"` or `"stamford"` in the first cell.
- **Runtime** → **Run all**. The setup loads the clean data and a **line plot** appears — it already saved a PNG. Read the short "how this was cleaned" note while it runs; that's your data dictionary.

2.3 Write your histogram (the one thing you code)

In the cell marked **TODO**, write the histogram. The shape is printed right above it — roughly:

```
ax = wx["temp_f"].plot.hist(bins=20, figsize=(8, 4), title="Most hours sit in the 60s and 70s")
ax.set_xlabel("temperature (F)")
ax.get_figure().savefig("weather_hist.png", dpi=150, bbox_inches="tight")
```

(Partner B: `dem["load_mw"]` → `demand_hist.png`.) Run it. **Look at it**. Retitle it with what a reader should notice. Keep the filename **exactly** as given.

2.4 Download your notebook and your PNGs

1. Run the **download** cell → your two PNGs land in your **Downloads** folder.
2. **File** → **Download** → **Download .ipynb** → your notebook lands in Downloads too.

💡 Colab *can* save straight to GitHub, but **we're not using that tonight** — you're moving the files in by hand so you see exactly where they live in the repo.

2.5 Drag everything into your repo

GitHub Desktop → **Repository** → **Show in Explorer** (Reveal in Finder on Mac). That's your repo folder. Make two folders — `notebooks/` and `images/` — then drag from **Downloads**: - your notebook (`.ipynb`) → `notebooks/` - your two PNGs → `images/`

⚠️ If a filename shows `(1)` or `(2)` — Downloads renamed it because you downloaded twice — **rename it back** to the exact name before dragging.

2.6 Commit and push

Back in **GitHub Desktop**: your notebook and PNGs are under **Changes**. Confirm the top bar says **your dev-branch**. Bottom-left: **Summary** = `add weather plots` → **Commit to dev-weather** → **Push origin**.

If it says **Pull origin** first, click it, then Push.

Part 3 — Review & merge (both, ~20 min)

3.1 Open your pull request

On github.com your repo shows a yellow bar: **Compare & pull request** → check it's `dev-weather` → `main` → title `Weather plots` → **Create pull request** → right side, **Reviewers** → your partner.

3.2 Review your partner's

Open your **partner's** PR → **Files changed**. Actually read it: their histogram cell (does the title say something? units on the axis?) and their two PNGs. GitHub won't unlock **Approve** until you **scroll** through the changes. Then **Review changes** → **Approve** → **Submit review**. Leave one real comment if you have one.

3.3 Merge, delete, pull

- **Merge pull request** → **Confirm** → **Delete branch**. Both PRs.
- **Both partners:** GitHub Desktop → **Current branch** → `main` → **Fetch origin** → **Pull origin**. Open `images/` in Explorer: **four PNGs**. Neither of you could have produced that alone.

Part 4 — Read-out & deliverable (~15 min)

4.1 Your deliverable — the network graph

On your repo: **Insights** → **Network**. The branches leaving `main` and coming back are your night — proof you both authored *and* reviewed. **Take a screenshot** and post it to **Lab 1 participation** on HuskyCT. That's what's due.

4.2 If you're ahead — a short report

Add a `REPORT.md` (github.com → **Add file** → **Create new file** → `REPORT.md`). Drop in your four plots with `!` [caption](images/weather_line.png) and write **one sentence** under each — every number gets a unit (°F, MW, hours). Commit it on a `report` branch → PR → the *other* partner approves → **Merge**. `main` is protected — that's the rule working.

4.3 If you're really flying — the joint plot

Open `notebooks/Lab1_Joint_Optional.ipynb` (same GitHub tab, class repo), run it → download `temp_vs_load.png` → drag into `images/` → commit on a branch → PR → merge. Chase the question at the bottom of that notebook — it's the surprise (the hottest hour is **not** the peak-demand hour).

When it breaks

Symptom	Fix
Push rejected on <code>main</code>	You're on <code>main</code> . Switch to your <code>dev-</code> branch and commit there. The rejection is the protection working.
Can't find the starter notebook	Colab → Open notebook → GitHub tab → paste <code>drdave-teaching/OPIM5512-labs</code> → <code>Module1/Week1_TechStack/Lab1_FirstCommit/notebooks/</code> .
"Should I use Colab's Save to GitHub?"	Not tonight. Download the <code>.ipynb</code> and drag it into <code>notebooks/</code> so you see where it lives.
Report/README shows a broken image	Filename mismatch — usually a (1) in the PNG name, or it's in the wrong folder. Exact name, in <code>images/</code> .
Deleted a branch by mistake	Merged PRs have a Restore branch button. Committed work is very hard to lose.
Fetch/Pull will overwrite my work?	No. Fetch peeks, Pull downloads. The scary buttons are the <i>Discard</i> ones. Commit first and you're safe.

Extended edition (you clean the data yourself, and stage a merge conflict on purpose): one folder up, [Lab1_instructions_opim5512.md](#). Those notebooks are in [notebooks/extended/](#).